

RELATÓRIO 03 — FECHO DE LACUNAS

M.I.N.D. — Muñdji Intelligent Neural Developer
Pontos 2 a 6 da ordem de trabalho

Campo	Valor
Data	3 de Agosto de 2026
Commit	7c6d807
Pull request	Jos021/021 #1 (aberto no início desta etapa)
Suite	203 passam · 4 saltados · nenhum xfail
Endpoints Ollama	Não disponíveis — a ordem escrita foi mantida, o piloto fica no ponto 6
Decisão do ponto 4	Unificação no LangGraph (teste técnico na secção 3)

1. Sumário

Fechei os pontos 2 a 6 da ordem de trabalho. Nenhuma lacuna foi fechada corrigindo documentação: cada uma foi implementada. Onde o método descrito não pôde ser seguido à letra, está dito na secção respectiva, com a razão.

A decisão que ficou em aberto — a dualidade LangGraph — foi resolvida com o teste técnico que pediste. **O LangGraph exprime o fluxo completo sem contorções**, portanto unifiquei lá e eliminei o orquestrador Python paralelo. Detalhe e prova reproduzível na secção 3.

2. Ponto 2 — Diff real na validação de contrato

Antes de distribuir, o CORTEX fotografa o conteúdo de todas as secções que não pertencem a cada NEURON. Quando o NEURON responde, as secções alheias são reconstruídas a partir da resposta e comparadas linha a linha com a fotografia. A heurística de marcadores mantém-se como primeiro filtro barato; o diff é a verificação autoritativa.

2.1 O caso que a heurística deixava passar

Um NEURON devolve a sua secção correctamente — com o seu marcador, sem marcadores alheios — e ao mesmo tempo altera o **preâmbulo partilhado** (imports, constantes). A heurística aprova; o diff apanha. É o teste `test_diff_apanha_alteracao_no_preambulo`, que começa por confirmar que a heurística sozinha deixaria passar.

2.2 Uma decisão de desenho, dita em voz alta

A primeira implementação reconstruía o ficheiro por concatenação de texto e acusava violação falsa quando o NEURON devolvia o preâmbulo *intacto* — o que é legítimo, porque cada NEURON recebe sempre o código base completo. Passei a reconstruir por secções: usa-se a versão que a resposta apresenta, e o que ela não menciona fica como estava.

Daí decorre uma regra deliberada: **uma secção alheia devolvida vazia conta como não mencionada, não como apagada**. O compromisso é consciente — apagar imports partilhados parte o código e é apanhado pelos testes da sandbox, ao passo que uma violação falsa queimaria um ciclo inteiro sem motivo. Está documentado na docstring da função.

3. Ponto 3 — `cleanup_temporary()` a sério

Limpeza real, com git a sério: branch órfão, reconstrução do histórico só com os commits permanentes por ordem cronológica, `git branch -M`, e `reflog expire + gc --prune=now` para tornar os objectos descartados fisicamente inalcançáveis. O teste `test_objectos_temporarios_deixam_de_ser_alcancaveis` confirma-o com `git cat-file`, porque sair do log não basta.

Aspecto	Como ficou
Método de reconstrução	<code>git cherry-pick</code> de cada permanente, como especificado. O primeiro commit não pode ser cherry-picked (não há sobre o que replicar um diff) e é materializado com <code>git commit-tree</code> . Se um cherry-pick entrar em conflito — possível ao replicar diffs de uma história a que faltam commits intermédios — recorre-se a <code>commit-tree</code> , que preserva a árvore exacta do commit aprovado. É plumbing de git, não um atalho.
Reescrita das tags	Passo que a especificação não previa mas sem o qual a limpeza seria apenas aparente: as tags <code>permanent_iter_N</code> antigas manteriam os commits originais alcançáveis e, através dos seus pais, também os temporários — o gc não poderia nada. As tags são apagadas e recriadas sobre os commits novos.
needs_human	A limpeza não corre. O nó <code>needs_human</code> não a invoca, e o teste <code>test_needs_human_nao_limpa_o_historico</code> confirma que o histórico completo fica.
Falha na limpeza	Aborta e deixa o histórico como estava. Uma limpeza falhada nunca pode destruir trabalho.

Implementar isto expôs uma lacuna maior do que a que eu tinha reportado. O MIND nunca escrevia o código no workspace durante as iterações — só na aprovação. O commit por iteração não tinha conteúdo nenhum e todo o versionamento era decorativo: o rollback e a intervenção manual em `needs_human` apoiavam-se num histórico vazio. Passou a escrever-se o código de cada iteração antes do commit, e a commitar por iteração mesmo quando a árvore não muda, porque o registo da percentagem atingida tem valor de auditoria por si só.

4. Ponto 4 — Dualidade LangGraph: teste técnico e decisão

Construí o fluxo real do MIND como *StateGraph* compilado e verifiquei, **em execução real e todas ao mesmo tempo**, as cinco decisões que a arquitectura não pode perder. O teste está versionado em *docs/decisoões/teste_tecnico_langgraph.py* e é reproduzível.

#	Decisão	Resultado
1	Rollback: iteração pior parte da melhor versão anterior	Exprime
2	Regra de divergência: >15pp força 3ª ronda	Exprime
3	Activação dinâmica: só os visados a partir da 2ª iteração	Exprime
4	Circuit breaker: timeout por NEURON sem bloquear o ciclo	Exprime
5	Limite de iterações com saída para needs_human	Exprime

Trajectória observada numa execução: `[['neuron_1', 'neuron_2'], ['neuron_2'], ['neuron_1'], ['neuron_1']]` — a 1ª passagem corre todos, a 2ª corre só o violador do contrato, as seguintes só o NEURON visado por melhoria.

4.1 Direcção tomada: unificação no LangGraph

O **LangGraph exprime tudo sem contorções**, portanto apliquei a primeira das duas direcções: o ciclo passa a correr sobre o grafo compilado e *MindGraph.run()* deixou de reimplementar o fluxo. Os métodos `_phase1`, `_phase2` e `_phase3` foram removidos. Existe uma única implementação executável.

As três acomodações necessárias não são distorções da lógica — são a forma natural de exprimir num grafo o que estava num ciclo *for*:

- **recursion_limit explícito.** Cada iteração atravessa cerca de uma dezena de nós e o valor por omissão (25) não chega. É um parâmetro documentado, calculado a partir de `MUNDJI_MAX_ITERATIONS`.
- **Nó nova_iteracao.** O contador passa a ser um nó em vez de uma variável de loop. Num grafo cíclico é onde deve estar.
- **Nó so_o_violador.** Já era lógica existente, inline no `run()`; apenas passou a ter nome próprio.

4.2 Dois falsos negativos do próprio teste

Registo-os porque ambos eram defeitos do teste, não do sistema, e ambos me teriam levado à conclusão errada se não os tivesse investigado:

- **O circuit breaker bloqueava o ciclo.** O NEURON de ensaio excedia o tempo em *todas* as rondas, o que é tratado como violação de contrato — correctamente — e o ciclo nunca chegava à Fase 3. Nessa versão o teste dizia que o LangGraph não exprimia nada, quando o problema era o ensaio.
- **O rollback é invisível a olho nu.** Quando a melhor versão é idêntica à actual, a substituição acontece na mesma mas não se vê comparando o código. Passou a ser detectado pelo registo na SYNAPSE DB, que é onde fica a prova.

5. Ponto 5 — Esquema JSON nos relatórios

O CEREBELLUM devolve a avaliação em JSON com esquema fixo (*functionality_pct*, *failures*, *improvements*, *auto_reject*). O prompt instrui explicitamente a responder só com esse objecto. O regex fica como recurso de último caso e, quando é usado, regista-se um aviso na SYNAPSE DB de que o modelo não respeitou o formato — informação directamente útil ao piloto, que é onde se vai descobrir que modelos cumprem o contrato.

Situação	Comportamento
JSON bem formado	Lido directamente; <i>functionality_pct</i> como número
JSON embrulhado em ``json ou em prosa	Extraído e lido, mas contabilizado como desvio ao formato
JSON válido sem <i>functionality_pct</i>	Falta o campo essencial: recorre ao regex
JSON malformatado / texto livre	Recorre ao regex e regista o aviso na SYNAPSE DB
<i>auto_reject</i> = true	Limita a percentagem abaixo do threshold — só pode reprovar
<i>auto_reject</i> = false	Não força aprovação nenhuma: a assimetria vale também aqui

Uma extensão que fiz além do pedido, e a razão. A instrução referia o CEREBELLUM. Apliquei o mesmo parser ao relatório do CORTEX, porque é o outro lado da mesma validação cruzada: ler um por JSON e o outro por regex deixaria metade do cálculo da percentagem na fragilidade que se queria eliminar — exactamente o tipo de meia-decisão que o ponto 3 da tua resposta criticava. Digo-o aqui em vez de o fazer em silêncio.

6. Ponto 6 — Decisões conscientes de não-fazer

Ficam registadas no código como decisões tomadas, não lacunas por resolver, à semelhança do que já existia para o Redis:

- **Isolamento do sandbox sem contentores** (*agent/sandbox.py*): o MIND corre o seu próprio código, não código hostil de terceiros. Caminho futuro documentado para o dia em que correr código não confiável.
- **Saída em ficheiro único** (*agent/graph.py*): a evolução natural — marcadores a transportarem o ficheiro de destino, [*NEURON_2:python:auth/tokens.py*] — fica descrita, com a nota de que o resto do sistema não precisaria de mudar.

7. Estado da suite

Ficheiro	N.º	Ficheiro	N.º
test_sanitizer.py	27	test_markers.py	17
test_hippocampus.py	24	test_sandbox.py	16
test_report_schema.py	23	test_cleanup.py	13
test_contract.py	20	test_ml_pipeline.py	13
test_database.py	20	test_dynamic_activation.py	9
test_cycle.py	18	test_regression_base.py	7

203 passam, 4 saltados (toolchains ausentes), nenhum xfail — o teste que documentava a lacuna do contrato foi substituído por sete testes reais do diff. Os nomes seguem a lista que indicaste. Uma nota de organização: a **promoção por comparação** está testada em *test_ml_pipeline.py* e não em *test_hippocampus.py*, porque é o pipeline que promove; o comportamento está coberto, só mora no ficheiro da sua própria área.

8. O que fica por fazer

#	Item	Estado
6	Piloto com modelos reais via Ollama e preenchimento de docs/model_selection_criteria.md com 5-10 tarefas por componente	Bloqueado: não há endpoints Ollama neste ambiente. Continua a ser a única coisa que valida tudo o resto — o sistema nunca correu com um LLM verdadeiro.
—	Saída multi-ficheiro e isolamento por contentores	Decisões conscientes de não-fazer, documentadas no código

Próximo passo assim que houver um endpoint Ollama: correr 5-10 tarefas reais por componente, medir quantas respostas respeitam o esquema JSON (a SYNAPSE DB já regista cada desvio), e preencher o critério de selecção de modelo com os resultados. Até lá não há mais nada de substancial que se possa validar sem simulação.

Relatório gerado no fim da etapa. Os resultados do teste técnico da secção 4 foram obtidos na execução versionada em docs/decisoese/teste_tecnico_langgraph.py.